

Architecture & Trade-offs

In-Memory Account Ledger Core · Mostafa Hanafy · 29 September 2026

Every behaviour cited here was reproduced against the code in `src/`. The timings come from a throwaway probe: N AED accounts, 50 credits each over 30 value days, one authorization per account, then one more `authorize` and one `endOfDay(30)`.

1. Append-only at scale

What breaks first: the end-of-day fee run. For every account and every day since Day 1, `endOfDay` calls `balance()`, which scans the *whole* entry log across all accounts, and `#hasFee`, which scans it again. The cost is $\text{accounts} \times \text{days elapsed} \times \text{total entries}$: quadratic in accounts, and it keeps growing even at flat volume.

Accounts	Entries	<code>endOfDay</code>	One <code>authorize</code>
100	5,000	49 ms	0.04 ms
1,000	50,000	9.8 s	0.36 ms
10,000	500,000	not run	5.8 ms

Ten times the accounts cost 200 times the time. At 100× volume, one day's fee run would not finish before the next day has to close.

Second to break: online authorization. `authorize` scans the full entry log (through `balance`) and every authorization ever recorded (through `activeHolds`). Its cost is linear in lifetime volume, and it has a hard deadline. If the issuer misses the card scheme's timeout, the scheme approves on its behalf (stand-in), and those approvals come back as force-posts. So a slow ledger becomes unchecked credit exposure.

Where state grows without bound

- **Entry log.** Append-only by design. That is fine for storage; the problem is that every read rebuilds balances from all of it.
- **Authorization log.** SETTLED and DECLINED records stay in the set `activeHolds` walks, and holds never expire.
- **Fee look-back.** Any value date is accepted, so `endOfDay` re-checks every day since Day 1, forever.
- **Error log.** It has no "resolved" state, and `#recordErrorOnce` dedupes it by linear scan.

Cheapest structural change: per-account projections, updated on append.

- The log stays the only source of truth. `#post` also updates, per account, three indexes: net movement per value day, the set of days already charged a fee, and a map of *open* holds.
- `balance` drops from $O(\text{all entries})$ to $O(\text{days for one account})$, and `activeHolds` and `#hasFee` become lookups.
- It is about 30 lines, changes no behaviour, and the existing tests prove that.
- It fixes compute, not memory. Bounding memory also needs a **back-value horizon**: nothing may be value-dated before it, so older days fold into an opening balance per account and move to cold storage. That horizon is a policy decision, and it is the control proposed in section 2.

2. Value-dated entries in production

Two dates per entry make restatement possible. In a UAE-licensed bank they also create this surface:

- **Customers.** A back-dated entry changes a balance the customer has already seen in a statement, the app or an SMS. `balance(account, day, knownBy)` can reproduce both views, but nothing records which one was sent. A fee value-dated three days back lands on a day the statement showed positive. Under the CBUAE Consumer Protection Regulation and Standards (fair treatment, fee transparency, complaints), a fee caused by the bank's own late booking is a predictable complaint and likely refund. That is exactly the case the failing test shows.
- **Credit bureau.** Overdraft status already reported to AECB can become wrong in either direction, so it needs a correction feed.

- **Finance and regulatory reporting.** An entry value-dated into a closed month or quarter changes the general ledger behind returns already filed with the CBUAE and behind IFRS statements. It must be booked as a prior-period adjustment or the period reopened.
- **Interest and profit.** A back-value across a capitalization date changes interest already paid (on Islamic accounts, profit already distributed). This design avoids the problem only because interest is capitalized once, at the end of the window.
- **Reconciliation.** Nostro and card-scheme settlement reports are matched on value date, so a wrong value date surfaces as a reconciliation break.
- **Financial crime and audit.** Back-valuing a credit earns interest on money the account never held, and back-valuing a debit can dodge a fee: classic insider-abuse patterns. Transaction monitoring that runs on booking date can miss patterns that only exist by value date. AML law requires records to be kept for five years, yet an `Entry` has no field for **who** posted it.

The control before go-live: a back-value limit with maker-checker.

- Any entry value-dated more than a set limit before its booked day, or into a closed period, is not auto-posted. It waits in a queue until a second authorised person approves it with a reason code.
- Before approving, the approver sees which days change sign and which fees, interest and issued statements are affected.
- Card clearing, where the scheme sets the value date, goes to reconciliation instead.
- In code, it is one check in `#post` plus an `actor` field on every entry. Finance sets the limit; I would not invent it.
- In this replay, E7 would have waited for a checker, who would have seen its three fees before any were charged.

3. Authorization lifecycle

The model has three statuses (APPROVED, DECLINED, SETTLED) and no expiry. A matching settlement has the same authorization ID, the same account and an amount within the hold. Every other ending is listed below.

#	How it ends in this model	Real-world scenario	Mandated behaviour
1	Declined at request. Terminal, no hold.	Insufficient funds (ISO 8583 response code 51).	Keep it terminal, but store the decision inputs (balance, holds, amount) so a dispute can show why. Today only the status is kept.
2	Duplicate ID. The second request returns DECLINED; the first record stands.	The network retransmits after a timeout.	Idempotent: return the <i>original</i> decision. Today, a resent approval returns DECLINED while the hold stays. Only a different payload under the same ID is a collision: reject it and alert.
3	Never ends. No expiry, so the hold blocks funds forever.	The merchant never clears it (for example, a hotel pre-authorization where the guest pays another way).	Expire by the scheme's limit for the merchant category by appending an EXPIRED record. A later settlement is still posted, under a late-presentment code.
4	Can't be voided. A reversal finds no entries (REVERSAL_TARGET_NOT_FOUND), so the hold stays.	The merchant cancels, or the terminal sends a timeout reversal (0400/0420).	An explicit, idempotent void that appends RELEASED. It must handle the void arriving before the authorization it cancels.
5	Settled on another account. Force-posted there and flagged; the original hold stays APPROVED.	The card was relinked, an account was migrated, or the acquirer sent bad data.	Link both sides in one case, and release the orphaned hold, so the customer is never both debited and held for the same purchase.
6	Settled twice. The second settlement is force-posted as SETTLEMENT_WITHOUT_AUTH.	Duplicate clearing, or legitimate split shipments.	Dedupe on the scheme's clearing reference. Multiple clearing flagged by the scheme draws down the remaining hold.
7	Settled after a decline. Force-posted and flagged.	The merchant forced the sale through.	Post it (the money has moved) and open a chargeback automatically.
8	Settled above the hold. SETTLED and flagged, with no tolerance.	Tips, currency conversion, fuel.	Keep flagging until the business sets a tolerance per merchant category.

4. What I cut and why

Cut	Why	Production risk it defers
In-memory only	Required by the brief	A crash loses the ledger. Durable append-only storage comes first.
Single writer	One synchronous replay	<code>authorize</code> checks, then <code>acts</code> , so two concurrent authorizations can both spend the same balance. Entry IDs come from the array length and collide with more than one writer.
No event idempotency	Each event arrives exactly once	Delivery is at-least-once in production, and a redelivered credit is booked twice (verified). Needs a unique key on the source event.
Integer days	No clock times in the brief	No cut-off time, time zone (Gulf Standard Time vs UTC) or business-day calendar (Saturday–Sunday weekend, public holidays).
Day closes on the next event	The stream is the only clock	A quiet day never closes. Production needs a scheduled end of day.
Customer side only, no general ledger	Account-level scope	Nothing to post against (fee income, interest expense, scheme settlement, suspense), so nothing reconciles to the general ledger.
Full-scan balances; minimal authorization lifecycle	Correct, and instant at 10 events	Sections 1 and 3.
Flat AED fee, no cap, no refund rule	The fee is given; a refund rule would be invented policy	Fee spirals, and the customer pays for the bank's own late booking (the failing test). Fees on non-AED accounts are flagged, never billed.
No overdraft limit	Posted debits aren't gated	Uncapped unarranged overdraft, so uncapped credit exposure.
One interest capitalization	One window	No day-count convention, tiers or recurring cycles, and <code>capitalizeInterest</code> throws if called twice.
No FX	The stream never mixes currencies	Cross-currency card settlements can't be represented.
Full reversals only, any value date	E9 is full and dated	No partial refunds. A reversal with the wrong value date leaves days negative and nothing flags it.
Errors as a flat log	The brief only asks for them to be printed	No owner, severity or deadline, so chargeback windows can expire unworked.
Bad input throws	Silently skipping bad data is worse	One bad message halts every account. Production needs a dead-letter queue.
No actor or authentication	The brief has no users	Nobody can answer "who posted this?", which the section 2 control needs.
Accounts fixed at start	Two accounts	No close, freeze or dormancy, so posting to a frozen account isn't blocked.